ENTERPRISE STANDARD OPERATING PROCEDURE (SOP)

Proxmox VE Cluster & Dual-Storage Implementation Guide

Complete Bare-Metal to 5-Node Scaled Production Deployment (Ceph + ZFS RAIDZ2 + HA)

Document Version: 2.0-Enterprise
Cluster Blueprint: 3 to 5 Physical Nodes
Author: Infrastructure Engineering
Release Date: September 18, 2026

1. Executive Summary & 5-Node Architectural Blueprint

This handbook provides the authoritative, end-to-end engineering blueprint for deploying a high-performance, fault-tolerant enterprise virtualization infrastructure on Proxmox Virtual Environment (PVE 8/9). The architecture incorporates two distinct, complementary storage tiers running concurrently across Dell PowerEdge / AMD EPYC servers with 10G/25G optical fabrics:

SHARED TIER: CEPH RBD
- **4 Disks / Node** (20 OSDs across 5 nodes)
- 3x distributed replication
- Zero-downtime VM live migration
- Automated self-healing & rebalancing

LOCAL TIER: ZFS RAIDZ2
- **7 Disks / Node** (RAIDZ2 per node)
- Double disk-failure tolerance
- Extreme direct NVMe/SAS throughput (4GB/s+)
- Cross-node snapshot replication (`pvesr`)

HIGH AVAILABILITY (HA)
- Hardware watchdog self-fencing
- Active CRM Master & 5 active LRMs
- Sub-minute VM automated failover
- Dedicated redundant corosync links

2. Hardware Bill of Materials (BOM) & Drive Mapping (Per Server)

Each physical node in the 5-node cluster is provisioned identically with 12 enterprise SAS drives (18.2 TB / 20 TB nominal Seagate Exos X20) and a dedicated hardware RAID1 boot pair:

SLOT / DRIVE ID	MODEL / SPECS	INTERFACE / SPEED	ASSIGNED SUBSYSTEM	REDUNDANCY / ALLOCATION
sdm (PERC H755)	2× 960GB Enterprise SATA SSD	Hardware RAID1 (BOSS/PERC)	Host OS / Proxmox VE	ext4 root / 32GB Swap / ISO Repository
sda, sdb, sdc, sdd	4× 18.2 TB Seagate Exos X20 SAS	12Gb/s SAS HBA Pass-through	Ceph Distributed Pool	4x Ceph OSDs (18.2 TB raw each; 3x Replica)
sde, sdf, sdg, sdh, sdi, sdj, sdk	7× 18.2 TB Seagate Exos X20 SAS	12Gb/s SAS HBA Pass-through	Local ZFS Pool (`zfs-ha`)	7-Disk RAIDZ2 (~91 TB usable; 2 parity disks)
sdl	1× 18.2 TB Seagate Exos X20 SAS	12Gb/s SAS HBA Pass-through	Global Hot Spare	Available for immediate ZFS/Ceph replacement

3. Network Architecture & 5-Node IP Allocation Scheme

The architecture implements strict network segmentation. Storage and migration traffic are isolated on dedicated 10G/25G optical interfaces (`nic3`) to prevent latency spikes on corosync heartbeat packets:

NODE NAME	NODE ID	MANAGEMENT IP (VMBR0 - 1G)	CEPH & MIGRATION IP (NIC3 - 10G)	COROSYNC LINK 0	COROSYNC LINK 1	CLUSTER ROLE
prox21	1	`10.31.6.171/22`	`192.168.10.21/24`	192.168.10.21 (Priority 1)	10.31.6.171 (Fallback)	Expansion Node
prox22	2	`10.31.6.172/22`	`192.168.10.22/24`	192.168.10.22 (Priority 1)	10.31.6.172 (Fallback)	Primary Leader / CRM Master
prox23	3	`10.31.6.173/22`	`192.168.10.23/24`	192.168.10.23 (Priority 1)	10.31.6.173 (Fallback)	Member Node / Ceph Mon
prox24	4	`10.31.6.174/22`	`192.168.10.24/24`	192.168.10.24 (Priority 1)	10.31.6.174 (Fallback)	Member Node / Ceph Mon
prox25	5	`10.31.6.175/22`	`192.168.10.25/24`	192.168.10.25 (Priority 1)	10.31.6.175 (Fallback)	Expansion Node

4. Phase 1: Bare-Metal Proxmox VE Installation & Hardening

Step 1.1: Server BIOS & Hardware RAID Configuration

1. **Boot Mode:** Set System BIOS to `UEFI Mode`. Disable Legacy BIOS.
2. **Virtualization Flags:** Enable `AMD-V`, `IOMMU (AMD-Vi)`, and `SR-IOV` in Processor Settings.
3. **RAID Controller Mode:**
   - Configure PERC H755/H740P into **HBA / Enhanced HBA Mode** for drives `sda` through `sdl`. Raw SAS disks must be exposed directly to the OS without hardware RAID encapsulation.
   - Configure `sdm` as a Hardware RAID1 Virtual Disk consisting of two 960GB M.2/SATA SSDs for the Proxmox VE OS.

> **CRITICAL SAS HBA SETTING:** Never expose disks to ZFS or Ceph as individual RAID0 virtual disks. Disk cache and write barrier controls must be handled natively by the ZFS and Ceph BlueStore drivers to prevent data corruption during unexpected power events.

Step 1.2: Proxmox VE OS Installation

1. Boot the server via iDRAC Virtual Media with the official Proxmox VE 8.x / 9.x ISO.
2. Select **Install Proxmox VE (Graphical)**.
3. **Target Harddisk:** Select `/dev/sdm` (PERC RAID1 OS volume). Click *Options* and verify: `filesystem: ext4`.
4. **Time & Keyboard:** Country: `India`, Time Zone: `Asia/Kolkata`, Keyboard: `U.S. English`.
5. **Management Network:**
   - Management Interface: `eno1` (Primary 1GbE onboard NIC)
   - Hostname: `prox22.cisai.local` (or appropriate node hostname)
   - IP Address: `10.31.6.172/22` | Gateway: `10.31.4.1` | DNS: `10.31.6.1`
6. Complete the installation and reboot the physical host.

Step 1.3: Network Interfaces Configuration (`/etc/network/interfaces`)

Edit `/etc/network/interfaces` on each server to define the primary bridge (`vmbr0`) and configure the 10GbE optical interface (`nic3`) with an MTU of 9000 (Jumbo Frames) for maximum Ceph and live migration throughput:

FILE CONFIGURATION: /ETC/NETWORK/INTERFACES — **TARGET: PROX22 (ADAPT IP FOR EACH NODE)**

```
auto lo
iface lo inet loopback

iface eno1 inet manual

auto vmbr0
iface vmbr0 inet static
        address 10.31.6.172/22
        gateway 10.31.4.1
        bridge-ports eno1
        bridge-stp off
        bridge-fd 0

# 10G Optical Interconnect for Ceph & Corosync High-Speed Fabric
auto nic3
iface nic3 inet static
        address 192.168.10.22/24
        mtu 9000
# Apply networking live without rebooting:
$ ifreload -a && ip a show nic3
```

Step 1.4: Base System Tuning, Chrony NTP & Repositories

Synchronize system clocks across all nodes (essential for Ceph and Corosync quorum) and enable the no-subscription enterprise package channels:

SHELL COMMANDS: REPOSITORIES & CLOCK SYNCHRONIZATION — **EXECUTE ON ALL NODES**

```
# 1. Disable enterprise repo and enable pve-no-subscription
$ rm -f /etc/apt/sources.list.d/pve-enterprise.list
$ echo "deb http://download.proxmox.com/debian/pve bookworm pve-no-subscription" > /etc/apt/sources.list.d/pve-no-sub.list
$ apt update && apt install -y chrony ifupdown2 ethtool sysstat iotop jq
# 2. Configure Chrony for ultra-low drift inter-node time sync
$ echo "server 10.31.6.1 iburst" >> /etc/chrony/chrony.conf && systemctl restart chronyd
$ chronyc tracking
```

5. Phase 2: Cluster Initialization & Multi-Node Join

Step 2.1: Initialize the Cluster on Leader (`prox22`)

Create the cluster on the designated leader node (`prox22`). Specify `link0` on the 10GbE network (`192.168.10.22`) for primary low-latency heartbeats, and `link1` on the 1GbE management network (`10.31.6.172`) for automatic failover:

SHELL COMMAND: CLUSTER CREATION WITH REDUNDANT COROSYNC LINKS — TARGET: PROX22

```
$ pvecm create cisai-pve --link0 192.168.10.22 --link1 10.31.6.172
$ pvecm status
Quorum information
------------------
Date:             Fri Sep 18 16:30:00 2026
Quorum provider:  corosync_votequorum
Nodes:            1
Node ID:          0x00000002
Ring ID:          2.1
Quorate:          Yes
```

Step 2.2: Join Additional Nodes (`prox23`, `prox24`, `prox21`, `prox25`)

Execute the join command from each joining node pointing to the cluster leader IP. Explicitly map both network links:

SHELL COMMANDS: JOINING CLUSTER NODES WITH DUAL-LINK BINDING — EXECUTED ON TARGET NODES

```
# Join prox23:
prox23$ pvecm add 192.168.10.22 --link0 192.168.10.23 --link1 10.31.6.173
# Join prox24:
prox24$ pvecm add 192.168.10.22 --link0 192.168.10.24 --link1 10.31.6.174
# When expanding to 5 nodes (prox21 & prox25):
prox21$ pvecm add 192.168.10.22 --link0 192.168.10.21 --link1 10.31.6.171
prox25$ pvecm add 192.168.10.22 --link0 192.168.10.25 --link1 10.31.6.175
```

Step 2.3: Verify Corosync Quorum & Redundant Links

VERIFICATION COMMANDS: COROSYNC QUORUM & NODE HEALTH — ANY NODE

```
$ pvecm status
Nodes:            3 (or 5)
Expected votes:   3 (or 5)
Total votes:      3 (or 5)
Quorum:           2 (or 3 for 5 nodes)
Quorate:          Yes
$ corosync-cfgtool -s
LINK ID 0: addr = 192.168.10.x, status = ring 0 active with no faults
LINK ID 1: addr = 10.31.6.x,     status = ring 1 active with no faults
```

> **VOTEQUORUM DYNAMICS (3 vs 5 NODES):** In a 3-node cluster, Quorum is maintained with **2 votes** (tolerates 1 node failure). In a 5-node cluster, Quorum is maintained with **3 votes** (tolerates 2 simultaneous node failures without losing cluster state or fencing).

6. Phase 3: Ceph Distributed Storage Implementation

Step 3.1: Install Ceph Packages & Initialize Cluster

Install Ceph Quincy/Squid/Tentacle natively on all nodes and bind Ceph daemons strictly to the high-speed 10G optical network (`192.168.10.0/24`):

SHELL COMMANDS: CEPH INSTALLATION & SUBNET INITIALIZATION — EXECUTED ON PROX22

```
# 1. Install Ceph binaries across nodes
$ pveceph install --version squid
# 2. Initialize Ceph cluster with dedicated public & cluster network
$ pveceph init --network 192.168.10.0/24 --cluster-network 192.168.10.0/24
```

Step 3.2: Deploy Ceph Monitors & Managers

Deploy Ceph Monitors across an odd number of nodes (3 monitors for a 3-node cluster; 3 or 5 monitors for a 5-node cluster) to preserve consensus:

SHELL COMMANDS: MONITOR & MANAGER PROVISIONING — EXECUTED FROM PROX22

```
$ pveceph mon create --mon-address 192.168.10.22
$ pveceph mon create --mon-address 192.168.10.23
$ pveceph mon create --mon-address 192.168.10.24
$ pveceph mgr create
```

Step 3.3: Disk Discovery & OSD Creation (4 Disks per Node)

Create 4 OSDs per server using the 4 dedicated SAS drives. Always reference persistent `/dev/disk/by-id/` paths to guarantee consistent device binding across kernel updates and drive re-enumerations:

SHELL COMMANDS: IDENTIFY WWN IDS & CREATE OSDS — EXECUTED ON EACH NODE

```
# 1. Inspect SAS drives by ID
$ ls -l /dev/disk/by-id/ | grep -E 'sd[a-d]$'
# 2. Wipe any legacy partition tables
$ for d in /dev/sda /dev/sdb /dev/sdc /dev/sdd; do wipefs -a $d; sgdisk --zap-all $d; done
# 3. Create OSDs via PVE Ceph CLI
$ pveceph osd create /dev/sda
$ pveceph osd create /dev/sdb
$ pveceph osd create /dev/sdc
$ pveceph osd create /dev/sdd
```

Step 3.4: Configure CRUSH Rules & Create 3x Replicated Pool

Create the primary shared RBD pool named `ceph` with a size of 3 (3x replica) and min_size of 2 (allows I/O when 1 copy is temporarily down):

SHELL COMMANDS: CEPH POOL CREATION & PVE DATACENTER BINDING — EXECUTED ON PROX22

```
# Create pool with 3x replication (32 PGs for 12 OSDs, 64 PGs for 20 OSDs)
$ pveceph pool create ceph --size 3 --min_size 2 --pg_num 32 --application rbd
# Enable PG autoscaler for automated capacity rebalancing
$ ceph osd pool set ceph pg_autoscale_mode on
# Verify cluster status
$ ceph -s
cluster: bcddd50d...
health:  HEALTH_OK
services: mon: 3 daemons, mgr: active, osd: 12 osds: 12 up, 12 in (or 20 for 5 nodes)
data:    pools: 2, pgs: 33 active+clean
```

7. Phase 4: ZFS RAIDZ2 Storage Implementation (7 Disks/Node)

Step 4.1: Retrieve Persistent Disk Identifiers

Identify the exact `/dev/disk/by-id/` WWN paths for drives `sde` through `sdk`:

SHELL COMMAND: IDENTIFYING 7 ZFS SAS DRIVES BY PERSISTENT ID — TARGET NODE

```
$ ls -l /dev/disk/by-id/ | grep -E 'sd[e-k]$' | awk '{print $9 " → " $11}'
```

Step 4.2: Create Enterprise ZFS RAIDZ2 Pool (`zfs-ha`)

Create the pool using 4K-aligned sectors (`ashift=12`), modern Zstandard compression, and POSIX extended attributes for maximum performance:

SHELL COMMAND: OPTIMIZED ZFS RAIDZ2 POOL CREATION — EXECUTED ON EACH NODE

```
$ zpool create -f -o ashift=12 \ -O compression=zstd \ -O atime=off \ -O xattr=sa \ -O normalization=formD \ -O
acltype=posixacl \ zfs-ha raidz2 /dev/sde /dev/sdf /dev/sdg /dev/sdh /dev/sdi /dev/sdj /dev/sdk
$ zpool status zfs-ha
  pool: zfs-ha
 state: ONLINE
 config:
        NAME        STATE     READ WRITE CKSUM
        zfs-ha      ONLINE       0     0     0
          raidz2-0  ONLINE       0     0     0
            sde     ONLINE       0     0     0
            sdf     ONLINE       0     0     0
            sdg     ONLINE       0     0     0
            sdh     ONLINE       0     0     0
            sdi     ONLINE       0     0     0
            sdj     ONLINE       0     0     0
            sdk     ONLINE       0     0     0
errors: No known data errors
```

Step 4.3: ZFS ARC Memory Sizing for High-Core Nodes

By default, ZFS allocates up to 50% of host RAM for the Adaptive Replacement Cache (ARC). On servers hosting memory-intensive production VMs, cap the ARC to 32GB to prevent out-of-memory kernel panics:

```
CONFIGURATION: LIMITING ZFS ARC CACHE (/ETC/MODPROBE.D/ZFS.CONF)                    ALL NODES
$ cat << 'EOF' > /etc/modprobe.d/zfs.conf # Cap ZFS ARC to 32GB maximum, 8GB minimum options zfs zfs_arc_max=34359738368
options zfs zfs_arc_min=8589934592 EOF
$ update-initramfs -u
```

Step 4.4: Register ZFS Storage & Configure Automated Replication (`pvesr`)

Register the pool in `/etc/pve/storage.cfg` and configure scheduled incremental snapshot replication every 30 minutes for sub-30min RPO protection:

```
SHELL COMMANDS: DATACENTER REGISTRATION & REPLICATION JOB CONFIGURATION                TARGET: PROX22
# 1. Add pool to PVE cluster storage
$ pvesm add zfspool zfs-ha --pool zfs-ha --content images,rootdir --sparse 1
# 2. Configure replication job for VM 9002 to replica node every 30 mins
$ pvesr create-job 9002-0 local/prox22 --schedule "*/30" --type local
$ pvesr status
JobID      Enabled    Target        LastSync             NextSync   Duration  FailCount State
9002-0     Yes        local/prox22  2026-09-18_16:30:04  17:00:00   5.84s     0         OK
```

8. Phase 5: High Availability (HA) & Live Migration

Step 5.1: Hardware Watchdog Configuration

Proxmox HA requires a hardware watchdog for self-fencing. If a node loses quorum or locks up, the hardware watchdog forcibly resets the node within 60 seconds so other nodes can safely recover its workloads:

```
SHELL COMMANDS: HARDWARE WATCHDOG ACTIVATION                          EXECUTED ON ALL NODES
# 1. Load Linux softdog module (or ipmi_watchdog on Dell iDRAC)
$ modprobe softdog
$ echo "softdog" >> /etc/modules
# 2. Verify watchdog active status
$ systemctl restart pve-ha-crm pve-ha-lrm
$ ha-manager status
quorum OK
master prox22 (active)
fencing armed (CRM watchdog active)
lrm prox22 (active, watchdog active)
lrm prox23 (active, watchdog active)
lrm prox24 (active, watchdog active)
```

Step 5.2: Configure HA Groups for Priority-Based Failover

Define HA groups to dictate failover precedence across the 5 nodes:

```
SHELL COMMANDS: HA GROUP DEFINITION & VM ENROLLMENT                    TARGET: PROX22
# Create high-priority group for production database workloads
$ ha-manager groupadd prod-group --nodes "prox22:100,prox23:80,prox24:60,prox21:40,prox25:20" --nofailback 1
# Enroll VM 9001 (Ceph shared storage) into HA
$ ha-manager add vm:9001 --group prod-group --max_restart 2 --max_relocate 2
```

Step 5.3: Verify Zero-Downtime Live Migration over 10G

```
SHELL COMMAND: TRIGGER LIVE MIGRATION & INSPECT QEMU TELEMETRY                 TARGET: PROX22
$ ha-manager migrate vm:9001 prox23
$ journalctl -u pve-ha-crm -n 15 --no-pager
service 'vm:9001': state changed from 'started' to 'migrate' (target = prox23)
migration downtime limit: 100 ms | transferred 726.4 MiB VM-state | average speed: 63.3 MiB/s
migration finished successfully | downtime: 23 ms | status: started
```

9. Phase 6: Step-by-Step Blueprint: Expanding from 3 to 5 Nodes

Expanding a live 3-node cluster to 5 physical nodes (adding `prox21` and `prox25`) must be performed without interrupting existing VMs or degrading storage:

Stage 1: Prepare Hardware & Network on Nodes 4 and 5

1. Install Proxmox VE base OS on `/dev/sdm` of `prox21` and `prox25`.
2. Configure `vmbr0` (`10.31.6.171` and `10.31.6.175`) and `nic3` (`192.168.10.21` and `192.168.10.25` with MTU 9000).
3. Verify network reachability using `ping -c 5 192.168.10.22`.

Stage 2: Join Nodes to Live Cluster

SHELL COMMANDS: JOINING NODES 4 AND 5 — **EXECUTED ON PROX21 & PROX25**

```
prox21$ pvecm add 192.168.10.22 --link0 192.168.10.21 --link1 10.31.6.171
prox25$ pvecm add 192.168.10.22 --link0 192.168.10.25 --link1 10.31.6.175
prox22$ pvecm status # Expected votes: 5, Quorum: 3
```

Stage 3: Expand Ceph Storage (Add 8 OSDs to Reach 20 OSDs)

1. Install Ceph binaries on new nodes: `pveceph install --version squid`
2. (Optional) Add Ceph Monitors on the new nodes if 5 monitors are desired:
   $ pveceph mon create --mon-address 192.168.10.21
   $ pveceph mon create --mon-address 192.168.10.25
3. Initialize the 4 SAS drives on `prox21` and `prox25` as OSDs:
   $ for d in /dev/sda /dev/sdb /dev/sdc /dev/sdd; do pveceph osd create $d; done
4. Ceph will automatically incorporate the 8 new OSDs and rebalance placement groups in the background.

Stage 4: Create ZFS RAIDZ2 Pool on New Nodes

SHELL COMMANDS: PROVISION ZFS-HA ON NEW NODES — **EXECUTED ON PROX21 & PROX25**

```
$ zpool create -f -o ashift=12 -O compression=zstd -O atime=off -O xattr=sa zfs-ha raidz2 /dev/sde /dev/sdf /dev/sdg /dev/sdh
/dev/sdi /dev/sdj /dev/sdk
```

Stage 5: Update HA Groups

SHELL COMMAND: EXPAND HA GROUPS TO 5 NODES — **TARGET: PROX22**

```
$ ha-manager groupset prod-group --nodes "prox22:100,prox23:80,prox24:60,prox21:40,prox25:20"
```

10. Phase 7: Real-World Troubleshooting & Disaster Recovery

DISASTER RECOVERY RULE: Never power-cycle multiple nodes simultaneously. In an emergency, always check quorum first before attempting storage or VM operations.

Playbook A: Ceph OSD Failure & Drive Replacement

Symptom: `ceph -s` reports `HEALTH_WARN: 1 osds down`. Drive indicator light shows solid amber.

PLAYBOOK: REPLACE FAILED CEPH OSD (E.G., OSD.3) — **CLUSTER TERMINAL**

```
# 1. Identify physical slot and WWN of failed OSD
$ ceph osd find 3
# 2. Stop and purge the failed OSD safely
$ systemctl stop ceph-osd@3
$ pveceph osd destroy 3 --cleanup 1
# 3. Physically swap the drive with a new 20TB SAS drive
# 4. Recreate the OSD on the new drive (e.g., /dev/sdb)
$ pveceph osd create /dev/sdb
$ ceph -s # Monitor PG backfill until HEALTH_OK
```

Playbook B: Corosync Loss of Quorum & Emergency Split-Brain Recovery

Symptom: Commands like `qm start` or `pvecm status` fail with `Cluster not ready - no quorum? (500)`. Occurs if 2 nodes fail in a 3-node cluster, or 3 nodes fail in a 5-node cluster.

```
PLAYBOOK: EMERGENCY SINGLE-NODE QUORUM RECOVERY                    SURVIVING NODE
# 1. Force the surviving node to consider itself quorate (emergency override)
$ pvecm expected 1
# 2. Inspect Corosync daemon status
$ systemctl status corosync
# 3. Once other physical nodes are repaired and booted, reset expected votes back:
$ pvecm expected 3 # (or 5 for 5-node cluster)
```

Playbook C: ZFS RAIDZ2 Drive Degradation & Replacement

Symptom: `zpool status zfs-ha` reports DEGRADED with read/checksum errors on a drive (e.g., `/dev/sdg`).

```
PLAYBOOK: REPLACE FAILED DISK IN ZFS RAIDZ2                        TARGET NODE
# 1. Locate physical drive slot via perccli
$ /usr/local/sbin/perccli64 /c0 /eall /sall show | grep -E 'Failed|Degraded|Offline'
# 2. Offline the failed drive in ZFS
$ zpool offline zfs-ha /dev/sdg
# 3. Physically hot-swap the SAS drive
# 4. Locate new drive identifier and trigger resilver
$ zpool replace zfs-ha /dev/sdg /dev/disk/by-id/scsi-35000c500b...
$ zpool status zfs-ha # Verify resilver progress
```

Playbook D: ZFS Replication (`pvesr`) Job Sync Failures

Symptom: `pvesr status` shows `FailCount: 1+` or snapshot lock errors.

```
PLAYBOOK: CLEAR REPLICATION LOCKS & FORCE INCREMENTAL SYNC          NODE HOSTING VM
# 1. Check replication log for specific failure reason
$ pvesr log --id 9002-0
# 2. If locked by stale process, destroy stale snapshot lock:
$ qm unlock 9002
# 3. Manually trigger synchronization job
$ pvesr run --id 9002-0 -v
$ pvesr status # Verify State: OK
```

Playbook E: 10G Optical Link Flapping & MTU Mismatch

Symptom: Ceph OSDs randomly drop or report high heartbeat latency. Live migration stalls at 99%.

```
PLAYBOOK: VERIFY JUMBO FRAMES & OPTICAL SIGNAL LEVELS                 BOTH NODES
# 1. Test 9000 MTU without fragmentation (-M do)
$ ping -M do -s 8972 -c 5 192.168.10.23
# 2. Inspect optical transceiver optical power (SFP+ DDM)
$ ethtool -m nic3 | grep -E 'Receiver signal average optical power|Tx power'
# 3. Inspect interface dropped packets / CRC errors
$ ip -s link show nic3
```

11. Phase 8: Daily Operational Cheat Sheet & Automation Script

Systems administrators can perform daily health audits across all subsystems using this consolidated command reference:

SUBSYSTEM	INSPECTION COMMAND	HEALTHY BASELINE OUTPUT	ACTION IF NON-COMPLIANT
Corosync Quorum	`pvecm status`	`Quorate: Yes` (All 3 or 5 votes present)	Run `corosync-cfgtool -s` to check link failure
Ceph Distributed Pool	`ceph -s`	`HEALTH_OK`, all OSDs up & in, PGs active+clean	Run `ceph health detail` to isolate degraded PGs
Ceph OSD Tree	`ceph osd tree`	All OSD weights = 1.0, status = UP	Restart failed daemon: `systemctl restart ceph-osd@ID`
ZFS RAIDZ2 Pools	`zpool status zfs-ha`	`state: ONLINE`, 0 errors across 7 disks	Initiate scrub: `zpool scrub zfs-ha`
HA Manager & CRM	`ha-manager status`	`master prox22 (active)`, `fencing armed`	Check journal: `journalctl -u pve-ha-crm -n 50`
ZFS Replication	`pvesr status`	`FailCount: 0`, `State: OK` on all jobs	Force sync: `pvesr run --id <JOBID> -v`
Network MTU / Latency	`ping -c 5 -i 0.2 192.168.10.x`	0.0% loss, avg latency < 0.10 ms	Verify switch port VLAN and optic seating
Automated Test Suite	`python3 run_tests.py --all`	`OVERALL RESULT: ALL TESTS PASSED`	Review generated report in `reports/` directory

12. Automated Validation Suite Execution

The integrated Python testing framework in this repository performs automated validation of all the above checks in under 90 seconds. To execute before maintenance windows or after cluster expansion:

```
EXECUTION: AUTOMATED MULTI-NODE TEST SUITE                    WORKSTATION / NODE
$ cd "/home/nishusaini/Desktop/Innspark/Proxmox cluster testing"
$ python3 run_tests.py --all
 >>> Running: CLUSTER Test Suite...       [PASS]
 >>> Running: NETWORK Test Suite...       [PASS]
 >>> Running: CEPH Test Suite...          [PASS]
 >>> Running: ZFS Test Suite...           [PASS]
 >>> Running: HA Test Suite...            [PASS]
 >>> Running: MIGRATION Test Suite...     [PASS]
OVERALL RESULT: ALL TESTS PASSED SUCCESSFULLY!
```

13. Engineering Approval & Document Sign-Off

Document Author:
Infrastructure Engineering
Innspark / CISAI Infrastructure Team
Signature: _____

Storage Architecture Review:
Ceph 20-OSD & ZFS RAIDZ2 Certified
10G/25G Dual-Subnet Fabric
Signature: _____

Management Authorization:
[ ] Approved for Production
[ ] Approved for Staging
Signature: _____